

กลยุทธ์การทดสอบ (Testing Strategies)

Software Testing Strategies & Methodologies

รายชื่อสมาชิกกลุ่มผู้จัดทำ

1. อรปรียา อาสนทอง (6605001699)
2. อภิญญา วงษ์ศรี (6605005344)
3. กีมทีนา ลี (6705006069)
4. สุธิดา ทรงรัมย์ (6605005716)
5. วิญญู สายแสง (6805007405)

บทนำและความหมายของการทดสอบ

หลักการพื้นฐาน มาตรฐานสากล ISO และแนวคิด Verification & Validation

ความหมายของการทดสอบซอฟต์แวร์ (Software Testing)

Software Testing คือกระบวนการตรวจสอบและประเมินระบบซอฟต์แวร์ เพื่อค้นหาข้อบกพร่อง (Defects) และยืนยันว่าระบบทำงานตรงตามข้อกำหนด

มาตรฐาน ISO/IEC/IEEE 29119: ควบคุมภาพรวมตั้งแต่แนวคิด กระบวนการจัดทำเอกสาร และเทคนิคการออกแบบ

- ค้นหาความผิดพลาดก่อนส่งมอบสู่ผู้ใช้งานจริง
- ลดความเสี่ยงด้านความปลอดภัยและความเสียหายทางธุรกิจ
- เพิ่มความมั่นใจในคุณภาพ แม้ไม่สามารถหา bug ได้ 100%

Quality Assurance

ตรวจสอบพฤติกรรมทั้งในสถานการณ์ปกติและกรณีสุดโต่ง พร้อมทั้งลดต้นทุนการแก้ไขข้อผิดพลาดในระยะยาว

Risk Reduction

ยืนยันความถูกต้องและความน่าเชื่อถือของระบบตามความต้องการของผู้มีส่วนได้ส่วนเสียทุกฝ่าย

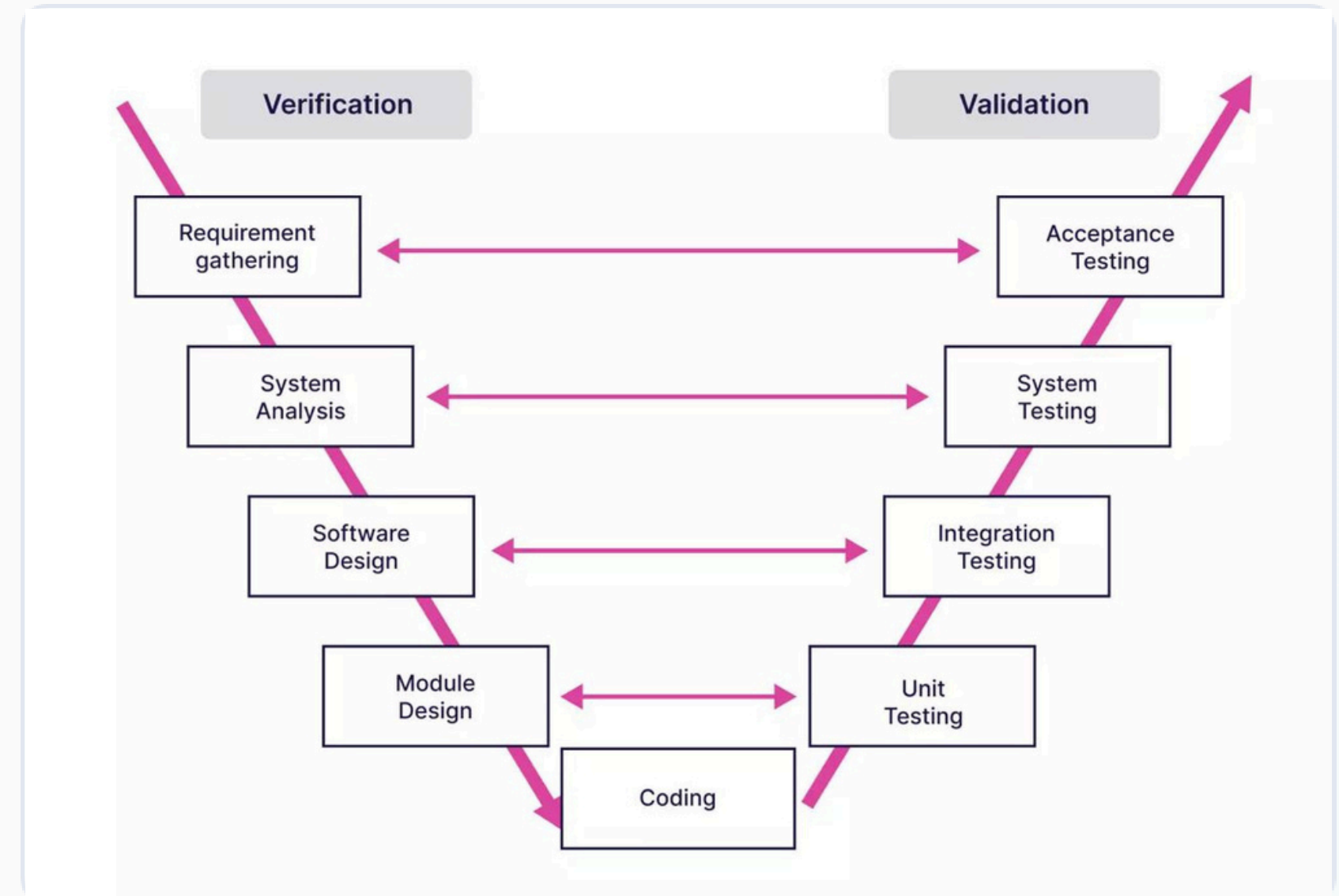
การตรวจทานและการตรวจรับ (V&V)

Verification: "เราสร้างผลิตภัณฑ์ถูกต้องตามข้อกำหนดหรือไม่?"

เน้นการตรวจสอบเอกสารและโค้ด เช่น Code Review, Design Review

Validation: "เราสร้างผลิตภัณฑ์ที่ถูกต้องสำหรับผู้ใช้หรือไม่?"

เน้นการทดสอบระบบจริงและความพึงพอใจของผู้ใช้งาน



ภาพแสดงความสัมพันธ์ระหว่าง Verification และ Validation ในวงจรพัฒนา

ข้อกำหนดการทดสอบและกรณีทดสอบ

Test Specification

เอกสารหลักที่ระบุแนวทาง ขอบเขต วัตถุประสงค์ และเกณฑ์การประเมินผลการทดสอบอย่างเป็นระบบ เพื่อให้ทีมงานดำเนินงานในทิศทางเดียวกัน

Test Case Elements

ประกอบด้วย Test ID, Preconditions, Input Data, Test Steps, Expected Results และ Actual Results เพื่อการตรวจสอบผลลัพธ์ที่แม่นยำ

TEST CASE EXAMPLE

Test case ID	Test scenario	Test steps	Expected result	Actual result	Pass/Fail
TC001	Login with valid credentials	1. Open login page 2. Enter valid username & password 3. Click login	User is successfully logged in	User is redirected to dashboard	Fail
TC002	Login with incorrect password	1. Open login page 2. Enter valid username & incorrect password 3. Click login	Error message appears	Error message appears	Pass



ตัวอย่างโครงสร้างข้อมูล Test Case ที่ใช้ในการทดสอบระบบจริง

กลยุทธ์เชิงวัตถุประสงค์และการทดสอบระดับหน่วย

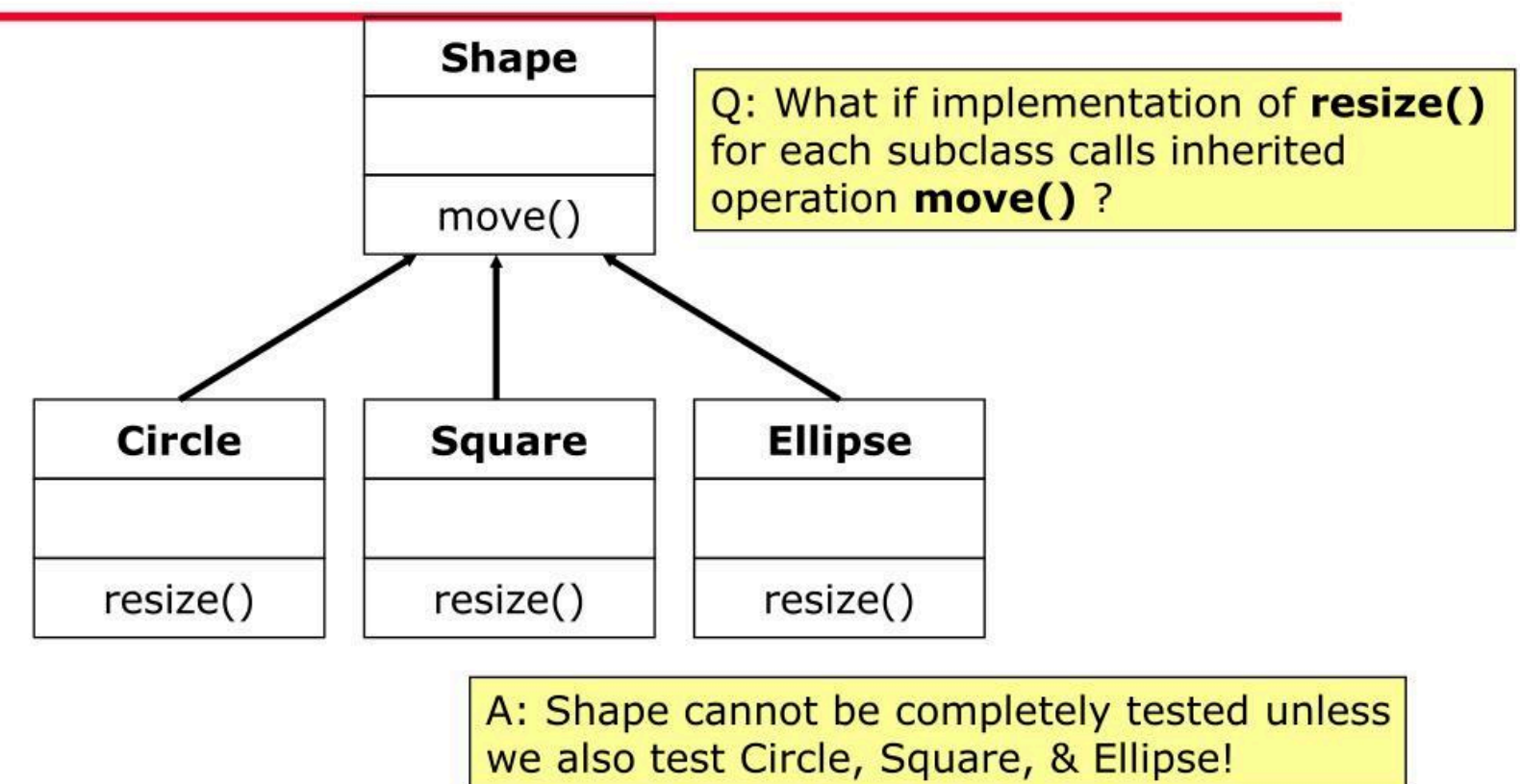
การทดสอบคลาส ออบเจกต์ และฟังก์ชันย่อยของระบบตามมาตรฐานสากล

กลยุทธ์เชิงวัตถุ (Object-Oriented Strategy)

การทดสอบระบบเชิงวัตถุมีความท้าทายสูงเนื่องจากคุณสมบัติเด่น ได้แก่ Encapsulation, Inheritance และ Polymorphism ซึ่งต้องคำนึงถึง:

- **Class และ Object:** ตรวจสอบว่าคลาสและ object ทำงานตามหน้าที่
- **Encapsulation:** ตรวจสอบข้อมูลและ operation ที่ถูกห่อหุ้มอยู่ใน class
- **Inheritance:** ตรวจสอบว่าคลาสลูกทำงานถูกต้องเมื่อรับคุณสมบัติจากคลาสแม่
- **Polymorphism:** ตรวจสอบว่าการเรียก operation ผ่านชนิดข้อมูลต่าง ๆ ให้พฤติกรรมที่ถูกต้อง
- **Object Interaction:** ตรวจสอบการทำงานร่วมกันระหว่าง object หลายตัว

Testing under Inheritance

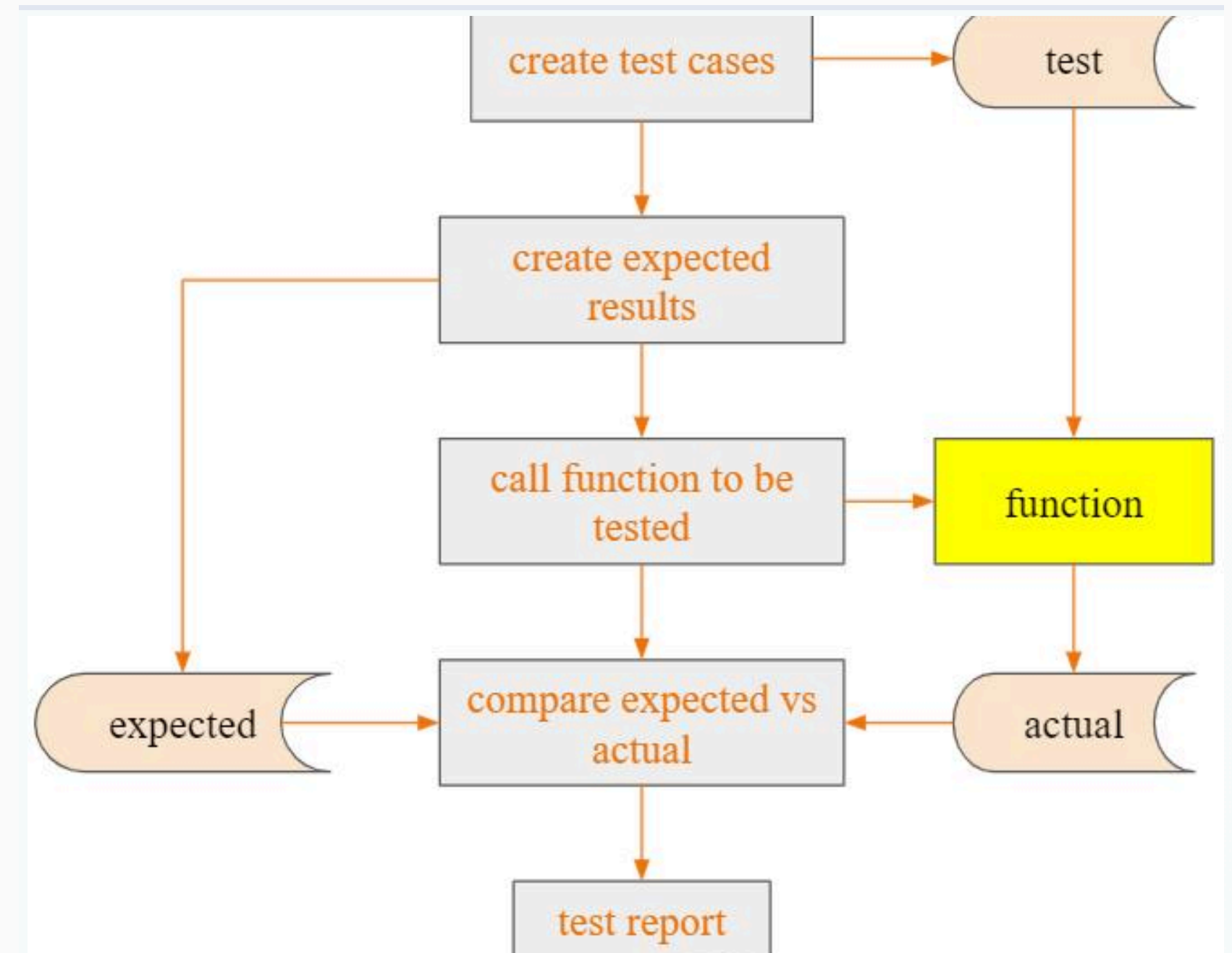


แผนภาพแสดงความสัมพันธ์ในการทดสอบคลาสและ
ออบเจกต์

การทดสอบระดับหน่วย (Unit Testing)

Unit Testing เป็นการทดสอบส่วนย่อยที่สุดของโปรแกรม เช่น Function, Method หรือ Class แยกจากส่วนอื่น ๆ เพื่อให้มั่นใจว่าโค้ดแต่ละส่วนทำงานถูกต้องแม่นยำ

- **Focus Area:** ตรวจสอบ Input/Output, ขอบเขตข้อมูล (Boundary values) และการจัดการข้อผิดพลาด
- **Real-world Example:** ฟังก์ชันคำนวณภาษีต้องทดสอบค่าต่ำกว่าเกณฑ์ เท่ากับเกณฑ์ และสูงกว่าเกณฑ์
- ดำเนินการโดยผู้พัฒนา (Developer) และนิยมเขียนแบบ Automated Tests



กระบวนการเขียนและรันชุดทดสอบระดับหน่วยแบบอัตโนมัติ

การทดสอบระบบและการบูรณาการ

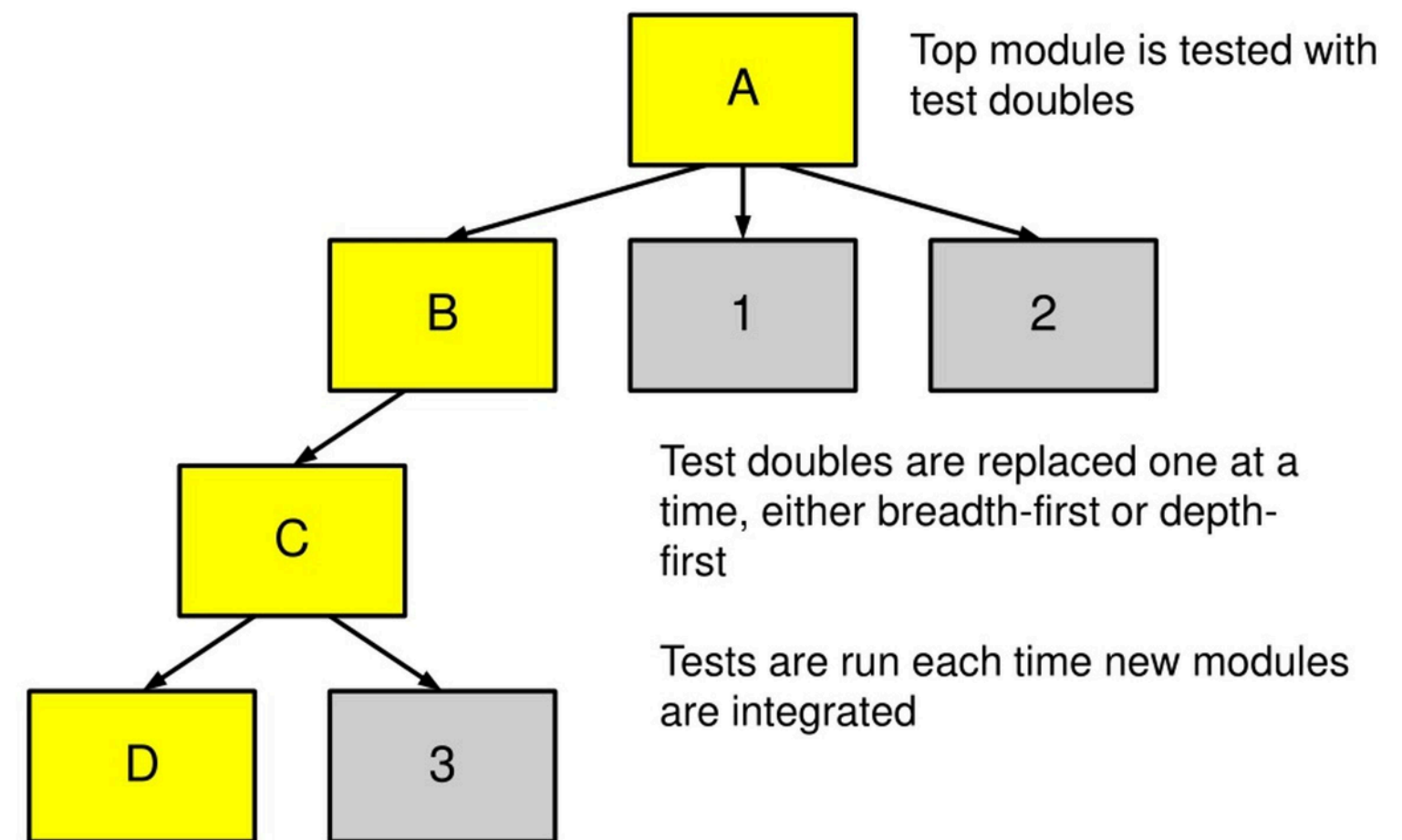
Integration Testing, Validation Testing และ
System Testing ขั้นสูง

การทดสอบระดับบูรณาการ (Integration Testing)

การนำ Unit ต่าง ๆ มาเชื่อมต่อกันเพื่อทดสอบการรับส่งข้อมูลและอินเทอร์เฟซ เพื่อค้นหาข้อบกพร่องที่อาจเกิดขึ้นจากการทำงานร่วมกันของระบบย่อย

- **Thread-based Testing:** ทดสอบตามลูปการทำงานหรือเหตุการณ์หลักของระบบ
- **Use-based Testing:** เริ่มจากคลาสอิสระไปสู่คลาสที่มีความซับซ้อนสูงขึ้น
- **Cluster Testing:** รวมกลุ่มคลาสที่เกี่ยวข้องเพื่อตรวจสอบความถูกต้องร่วมกัน

Top-Down Integration Testing



ตัวอย่างสถาปัตยกรรมการทดสอบแบบ Integration

Validation และ System Testing

Validation Testing

ตรวจสอบว่าซอฟต์แวร์ตรงตามความต้องการทางธุรกิจ (Business Requirements) ของลูกค้าและผู้ใช้งานจริงหรือไม่ โดยเน้นมุมมองผู้ใช้เป็นหลัก

- ทดสอบ Scenario การใช้งานจริง เช่น ระบบจองบริการ
- เน้นการทำ Acceptance Testing ร่วมกับลูกค้า

System Testing

ทดสอบภาพรวมของระบบทั้งหมดทั้งในส่วน Functional และ Non-functional requirements เช่น ประสิทธิภาพและความปลอดภัย

- ตรวจสอบ Performance, Security และ Reliability
- ทดสอบ Workflow ตั้งแต่ต้นจนจบกระบวนการ

การดีบั๊ก (Debugging)

Debugging คือกระบวนการค้นหาสาเหตุของ failure หรือ defect ที่พบจากการทดสอบ แล้วแก้ไขสาเหตุที่ทำให้โปรแกรมทำงานผิดพลาด

ขั้นตอนการ Debugging

- Class และ Object: ตรวจสอบว่าคลาสและ object ทำงานตามหน้าที่
- Encapsulation: ตรวจสอบข้อมูลและ operation ที่ถูกห่อหุ้มอยู่ใน class
- Inheritance: ตรวจสอบว่าคลาสลูกทำงานถูกต้องเมื่อรับคุณสมบัติจากคลาสแม่
- Polymorphism: ตรวจสอบว่าการเรียก operation ผ่านชนิดข้อมูลต่าง ๆ ให้พฤติกรรมที่ถูกต้อง
- Object Interaction: ตรวจสอบการทำงานร่วมกันระหว่าง object หลายตัว

Debugging แตกต่างจาก Testing คือ Testing มีหน้าที่ค้นหาและแสดงให้เห็นว่ามีปัญหา ส่วน Debugging มีหน้าที่ค้นหาสาเหตุและแก้ไขปัญหา

สรุปภาพรวมของ Testing Strategies

การทดสอบซอฟต์แวร์ควรดำเนินการเป็นหลายระดับ ไม่ใช่พึ่งการทดสอบเพียงประเภทเดียว โดยสามารถมองเป็นลำดับจากขนาดเล็กไปใหญ่ได้ดังนี้

ลำดับขั้นตอนการทดสอบมาตรฐาน

Unit Testing → Integration Testing → Validation → System Testing

Testing: ค้นหาและชี้จุดบกพร่องของระบบ

Debugging: วิเคราะห์หาสาเหตุและแก้ไขโค้ด

แหล่งอ้างอิง

- 1.ISO. ISO/IEC/IEEE 29119-1:2022 — Software and systems engineering — Software testing — Part 1: General concepts. อธิบายแนวคิดและคำศัพท์พื้นฐานของ software testing [1].
- 2.ISO. ISO/IEC/IEEE 29119-2:2021 — Software and systems engineering — Software testing — Part 2: Test processes. อธิบายกระบวนการสำหรับการวางแผน จัดการ และดำเนินการทดสอบ [2].
- 3.ISO. ISO/IEC/IEEE 29119-3:2021 — Software and systems engineering — Software testing — Part 3: Test documentation. อธิบายเอกสารและ template สำหรับกิจกรรมการทดสอบ [3].
- 4.ISO. ISO/IEC/IEEE 29119-4:2021 — Software and systems engineering — Software testing — Part 4: Test techniques. อธิบายเทคนิคการออกแบบการทดสอบ [4].
- 5.Pressman, R. S. & Maxim, B. R. Software Engineering: A Practitioner's Approach. แนวคิดเกี่ยวกับ testing strategies, verification and validation, unit testing, integration testing, validation testing, system testing และ debugging.
- 6.Sommerville, I. Software Engineering. แนวคิดเกี่ยวกับ software testing, verification and validation, test levels และ debugging.
- 7.เอกสารประกอบการสอนหัวข้อ “Testing Strategies” ซึ่งอธิบาย Verification and Validation และกลยุทธ์การทดสอบสำหรับซอฟต์แวร์เชิงวัตถุ รวมถึง Thread-based, Use-based และ Cluster Testing [5].